

- 1. **Agent** - Anything that can perceive its environment through sensors and acting upon that env. through actuators
- **Agent Function** - Maps from percept histories to actions
- **Rational Agent** - Chooses an action that is expected to maximize its performance measure, given by percept sequence and built-in knowledge
- **Autonomous Agent** - If behavior is determined by its own experience

- 1.1 **Performance Measure of Function**
- Motivation: For an agent to do the right thing, need a measure of goodness

1.2 Defining the Problem: PEAS

- Performance measure
- Environment
- Actuators
- Sensors

1.3 Characterizing the Environment

- Fully observable** - (vs. Partially) Agent's sensors can access complete state of env. all the time
- Deterministic** - (vs. Stochastic) Next state of env. is determined by **current state** and **action executed by agent**
 - **Strategic** - If env. is deterministic except for actions of other agents
- Episodic** - (vs. Sequential) Agent's experience is divided into atomic **episodes**, where each episode includes perceiving and an action, and **action depends on episode** itself
- Static** - (vs. Dynamic) Env. is unchanged while agent is deciding
 - **Semi** - Time does not affect env., but affects performance score
- Discrete** - (vs. Continuous) Discrete num. of percepts and actions
- Single Agent** - (vs. Multi-agent) Agent operating by itself in an env.

1.4 Implementing Agents (in ascending complexity)

- Simple Reflex Agents** - Fixed conditional rules
- Model-based Reflex Agents** - Stores percept history to make decisions about internal model of world with conditional rules. Eg. Roomba
- Goal-based Agents** - Keep in mind a goal and action aims to achieve it
- Utility-based Agents** - Find best way to achieve goal
- Learning Agents** - Learn from previous experiences

1.5 Exploitation vs. Exploration

- **Exploitation** - Maximize expected utility using current knowledge of world
- **Exploration** - Learn more about the world to improve future gains. May not always maximize performance measure.

2 02. Uninformed Search

- Deterministic, fully observable
- **Tree Search** - Can revisit nodes
- **Graph Search** - Tracks visited (Tree Search + Memoization)
- **Uninformed Search** - Uses only information available in problem definition

2.1 Formulating the Problem

- How to represent state in problem?
 - Initial state
 - Actions: Successor function
 - Goal test
 - Path cost
- **Abstraction Function** - Maps abstracted representation to real world state
 - **Representation Invariant** - $I(c) = \text{True} \rightarrow \exists a \text{ s.t. } AF(c) = a$

2.2 Breadth-first Search

- Idea: Expand shallowest unexpanded node using **queue**
- Given: b : Branching factor and d : Depth of optimal solution
- Complete: Yes (if tree is finite)
- Time: $O(b^d)$
- Space: $O(b^d)$
- Optimal: Yes (if cost = 1)
- BFS is Uniform-cost Search with same cost

2.3 Uniform-cost Search

- Idea: Expand least-cost unexpanded node using **priority queue** (Dijkstra's)
- Given: C^* : Cost of optimal solution
- Complete: Yes (if step cost $\geq \epsilon$ where $\epsilon \geq 0$)
- Time: $O(b^{(C^*/\epsilon)})$ (C^*/ϵ is approx. number of layers)

- Space: $O(b^{(C^*/\epsilon)})$

- Optimal: Yes

2.4 Depth-first Search

- Idea: Expand deepest unexpanded node using **stack**
- Given: m : Maximum depth of tree
- Complete: No (fails with infinite depth or loops)
- Time: $O(b^m)$
- Space: $O(bm)$ (better than BFS)
- Optimal: No

2.5 Depth-limited Search

- Motivation: How to handle infinite depth for DFS?
- Idea: DFS with depth limit l where nodes at depth l have no children
- Time: $b^0 + b^1 + \dots + b^{(d-1)} + b^d = O(b^d)$

2.6 Iterative Deepening Search

- Motivation: How to determine depth limit? We don't.
- Idea: Try different depths for depth-limited search
 - BFS pretending to be DFS to save space
- Complete: Yes
- Time: $(d + 1)b^0 + db^1 + \dots + b^d = O(b^d)$ (More overhead than DLS)
- Space: $O(bd)$

2.7 Summary

	BFS	Uniform Cost	DFS	DLS	IDS
Complete	Yes	Yes	No	No	Yes
Time	$O(b^d)$	$O(b^{C^*/\epsilon})$	$O(b^m)$	$O(b^l)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{C^*/\epsilon})$	$O(bm)$	$O(bl)$	$O(bd)$
Optimal	Yes	Yes	No	No	No

2.8 Bidirectional Search

- Idea: Search both forwards from initial state and backwards from goal state. Stop when searches meet.
- Time: $O(2b^{d/2})$
- Operators must be reversible
- Can have many goal states
- How to check if node intersects with other half?

3 03. Informed Search

3.1 Heuristic

- **Heuristic** - Estimated cost from n to goal
- **Admissible** - $h(n)$ is admissible if, for every node n , $h(n) \leq h^*(n)$ where h^* is the true cost
 - if h is admissible, then A* using tree search is optimal
- **Consistent** - $h(n)$ is consistent if, for every node n and every successor n' of n generated by action a , $h(n) \leq c(n, a, n') + h(n')$
 - Triangle inequality
 - If h is consistent, $f(n)$ is non-decreasing along any path ($f(n') \geq f(n)$)
 - If h is consistent, then h is admissible
 - if h is admissible, then A* using graph search is optimal

3.1.1 Dominance

- If $h_2(n) \geq h_1(n)$ for all n , then h_2 **dominates** h_1
- If h_2 **dominates** h_1 and both are admissible, then h_2 is better for search

3.1.2 How to invent admissible heuristic?

- Set fewer restrictions on actions
- E.g. Number of misplaced tiles, Total manhattan distance

3.2 Best-first Search

- Idea: Expand most desirable node using priority queue
- Evaluation Function: $f(n) = h(n)$
- Complete: No. Possible to be stuck in loop
- Time and space: $O(b^m)$
- Optimal: No

3.3 A* Search

- Idea: Take note of cost so far and heuristic
- Evaluation Function: $f(n) = g(n) + h(n)$ where $g(n)$ is cost to reach n
- Complete: Yes, unless non-increasing, since cost is factored in
- Time and space: Same as BFS
- Optimal: Yes, depending on the heuristic

3.4 Iterative Deepening A* Search (IDA*)

- Motivation: How can we save space?
- Idea: Have a cutoff for f and remember the best f that exceeds cutoff
 - Similar to IDS. Linear space complexity.
- Optimal and complete

3.5 Simplified Memory A* Search (SMA*)

- Motivation: How can we save space?
- Idea: Do normal A*. If memory is full, drop node with worst f .

- Lose completeness

3.6 Local Search

- Motivation: What if the goal state is the solution? The path is irrelevant.
- Idea: Keep single current state and try improving it
- Formulating the problem:
 - Initial state
 - Actions: Successor function
 - Good heuristic
 - Goal test

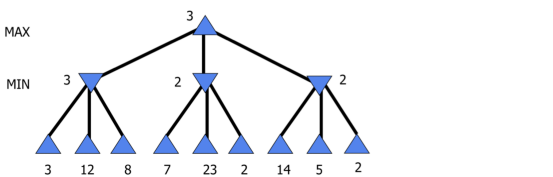
3.6.1 Hill-climbing

- Idea: Generate successors from current and pick the best using heuristic
- What if stuck into local minima?
 - Introduce randomness
 - **Simulated Annealing Search** - Allow some bad moves and gradually decrease frequency
- Why only keep 1 best state?
 - **Beam Search** - Perform k hill-climbing searches in parallel
- How to generate successors?
 - **Genetic Algorithms** - Successor is generated by combining 2 parent states

4 04. Adversarial Search

- Assumption: Opponents reacts rationally
- Formulating the problem:
 - Initial state
 - Successor function
 - Terminal test
 - Utility function: Measures how good the move is for a player

4.1 Minimax



- Idea: Choose move that yields highest minimax value using DFS
- Complete: Yes (if tree is finite)
- Time: $O(b^m)$
- Space: $O(bm)$
- Optimal: Yes (against an optimal opponent)

4.1.1 How to Draw a Game Tree?

- Do not have repeated states **per level**

4.2 Alpha-Beta Pruning

- Motivation: How to save time for Minimax?
- Idea: By tracking max. and min. values so far, can prune some paths that we would never choose
 - α contains max. and β contains min.
 - Initially, $\alpha = -\infty$ and $\beta = \infty$
 - When going down, copy α and β
 - Prune if $\alpha \geq \beta$
 - When going up, depending on MIN/MAX level, update α or β
- With perfect ordering, time complexity: $O(b^{m/2})$. Doubles search depth.

4.3 Resource Limits

- In reality, search space for games can be very large. α - β pruning also not fast enough.
- Solution: Limit depth (Only see a finite moves ahead) and determine best move using evaluation function to estimate desirability of position (Heuristic)
- Other hacks:
 - **Transpositions** - Memoize equivalent states
 - Pre-computation of opening/closing moves

5 05. Introduction to Machine Learning

- A machine learns if it improves performance P on task T based on experience E. Where T must be fixed, P must be measurable, E must exist

5.0.1 Types of Feedback

- **Supervised** - Correct answer given for each example
 - **Regression** - Predict results within continuous output
 - **Classification** - Predict results in discrete output
- **Unsupervised** - No answers given
- **Weakly supervised** - Answer given, but not precise
- **Reinforcement** - Occasional rewards given

5.1 Decision Trees

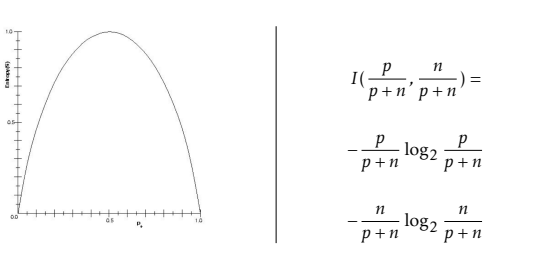
- DT can express any function of input attributes, if data is consistent
- Goal: Make DT **compact**. How?

5.1.1 Information Theory

- Idea: Choose attribute that splits examples into subsets that are ideally 'all positive' or 'all negative'
- **Entropy** - Measure of randomness in set of data

$$I(P(v_1), \dots, P(v_n)) = - \sum_{i=1}^n P(v_i) \log_2 P(v_i)$$

- For data with p positive examples and n negative examples:



- **Information Gain** - (IG) Reduction in entropy from attribute test
- Goal: Choose attribute with largest information gain
- Intuition: IG = Entropy of this node - Entropy of children nodes
- Given chosen attribute A with v distinct values:

$$\text{remainder}(A) = \sum_{i=1}^v \frac{p_i + n_i}{p + n} I\left(-\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right)$$

$$IG(A) = I\left(-\frac{p}{p+n}, \frac{n}{p+n}\right) - \text{remainder}(A)$$

- **Decision Tree Learning** - Recursively choose attributes with highest IG
- IG is not the only way. Can use whatever objective function that achieves the criteria we want.

5.2 Performance Measurement

- **Correctness** - Correct if $\hat{y} = y$

- **Accuracy** - $\frac{1}{m} \sum_{j=1}^m (\hat{y}_j = y_j)$

- Confusion Matrix:

		Actual Label	
		+ve	-ve
Predicted Label	+ve	TP True Positive	FP False Positive
	-ve	FN False Negative	TN True Negative

- Accuracy = $\frac{TP+TN}{TP+FN+FP+TN}$

- **Precision** - $\frac{TP}{TP+FP}$ How precise are positive predictions?

- **Recall** - $\frac{TP}{TP+FN}$ How many actual positives are predicted?

- **F1 Score** - $\frac{2}{1/P+1/R}$ Harmonic mean of precision and recall

- Type I Error: FP, Type II Error: FN
- FP Rate = $\frac{FP}{FP+TN}$ TP Rate = $\frac{TP}{TP+FN}$

5.3 Pruning

- Motivation: DT overfits to training set, but performs poorly on test set
- Occam's Razor: Simple hypothesis preferred
- **Pruning** - Ignores outliers, which reduces overfitting
 - Idea: Go with the majority of T/Fs
 - E.g. Min-sample, Max-depth

6 Uncategorized

6.1 α - β -pruning

- init $\alpha = -\infty, \beta = +\infty$
- MAX node: update & return $\alpha(\max)$, prune the rest if $\alpha \geq \beta$
- MIN: update & return $\beta(\min)$, prune the rest if $\beta \leq \alpha$

- 6.2 Decision Trees**
- if they asking for which attribute chosen first, just compare columns try to match.
 - the more pure groups the better
 - Impure large groups affect IG more than Impure small groups.
 - **Entropy:**
 - 0 if certain, 1 if completely uncertain
 - $H(1, 0)/H(0, 1) = 0|H(0.5, 0.5) = 0$
 - $H(Y) = H(P(y_1) \dots P(y_k)) = -\sum_{i=1}^k P(y_i) \log_2 P(y_i)$
 - e.g. $-(\frac{1}{6} \log_2 \frac{1}{6} + \frac{1}{3} \log_2 \frac{1}{3} + \frac{1}{2} \log_2 \frac{1}{2})$
 - **Specific Conditional Entropy:**
 - $H(Y | X = x) = H(P(y_1 | x), \dots, P(y_k | x))$
 - e.g. given x = some value, find the entropy
 - **Conditional Entropy:**
 - $H(Y | X) = \sum_{x \in X} [P(X = x) H(Y | X = x)]$
 - given specific entropies H_i and probability p_i of each given $x = i$,
 - Cond Ent = Sum of all $p_i \times H_i$
 - **Information Gain:**
 - $IG(Y, X) = H(Y) - H(Y | X)$
 - **IG for subnodes:**
 - IG = Entropy of node - Sum of weighted entropies of groups in the node
 - **Remaining Entropy at any point during attribute selection:**
 - $\sum$ weighted entropies in impure groups

Linear & Logistic Regression Cheatsheet

Linear Regression Basics

Model: $h_w(x) = w_0 + w_1 x_1$ (bias w_0 , or use dummy $x_0 = 1$).

MSE Loss: $L_{MSE}(w) = \frac{1}{2n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)})^2$

Example (6A): $w = [2, 1]^T$, data: (0, 0), (2, 1), (3, 2)

$h(x^{(1)}) = 2 + 0 = 2$, $h(x^{(2)}) = 2 + 2 = 4$, $h(x^{(3)}) = 2 + 3 = 5$

$L_{MSE} = \frac{1}{6} [2^2 + 3^2 + 3^2] = \frac{22}{6} = \frac{11}{3}$

SGD Update (6B)

Pick random point i . Update:

$$w \leftarrow w - \alpha \cdot (h_w(x^{(i)}) - y^{(i)}) \begin{bmatrix} 1 \\ x^{(i)} \end{bmatrix}$$

Example: Random point $i = 2$, $\alpha = 1/3$.

Residual = $h(x^{(2)}) - y^{(2)} = 4 - 1 = 3$

$$w' = \begin{bmatrix} 2 \\ 1 \end{bmatrix} - \frac{1}{3}(3) \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 2 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

Linear Dependence & Convexity (6C)

If features are linearly dependent (e.g. $x_2 = 3x_1$): loss is convex (not strictly), so GD may converge to *different weights* but *same loss*. Independent features $\Rightarrow$ strictly convex $\Rightarrow$ unique weights. **Key:** same loss, possibly different weights. $A : x \rightarrow h^A, x' \rightarrow h^{A'}$ $B : x \rightarrow h^B, x' \rightarrow h^{B'}$ (training on x (linearly dependent) may produce different weights but same loss. training on x' (linearly independent) will produce same weights same loss

Feature Scaling for Stability (6D)

When features have very different scales (e.g. $x_3 \sim 1000$ vs others ~ 1):

- **Standardization:** $(x - \mu)/\sigma$
- **Min-max scaling:** $(x - x_{min})/(x_{max} - x_{min})$
- Normal equation fails if features are linearly dependent (non-invertible $X^T X$)

Logistic Regression (6E)

Use $\sigma(z) = 1/(1 + e^{-z})$ when predicting *probability*.

Need bias w_0 if $P > 0.5$ when all $x_i = 0$ (since $\sigma(0) = 0.5$, need $w_0 > 0$).

Write as $P(Y|X) = \sigma(w_0 + w_1 x_1 + w_2 x_2 + w_3 x_3)$. Note: $P(Y|X)$ not $P(Y|w)$.

Normal Equation (6F)

$$w = (X^T X)^{-1} X^T y$$

Diagonal matrix inverse: flip each diagonal entry $d_i \rightarrow 1/d_i$.

Example: $X = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, y = \begin{bmatrix} 5 \\ 6 \end{bmatrix}$

$X^T X = \begin{bmatrix} 1 & 0 \\ 0 & 4 \end{bmatrix}, (X^T X)^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1/4 \end{bmatrix}$

$X^T y = \begin{bmatrix} 5 \\ 12 \end{bmatrix}, w = \begin{bmatrix} 1 & 0 \\ 0 & 1/4 \end{bmatrix} \begin{bmatrix} 5 \\ 12 \end{bmatrix} = \begin{bmatrix} 5 \\ 3 \end{bmatrix}$

6.2.1 Inverting 2x2 Matrix

For $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}, A^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$